

AMDP 練習 3：SQLScript 基本語法——變數、流程控制

Lecture

am01/am02 的 SQLScript 程序內容都是「宣告式」的：整段方法本體就是一句（或兩句平行的）**SELECT**，把查詢結果直接指派給 **EXPORTING** 參數，跟 Open SQL 的 **SELECT ... INTO TABLE** 用起來的思維很接近，只是跑在資料庫引擎裡。但 SQLScript 其實是一個完整的程序語言，除了宣告式查詢，還支援**命令式**（**procedural**）的寫法：宣告區域變數、用 **IF/CASE** 做條件分支、用 **FOR/WHILE** 逐筆處理資料。這題示範這一塊。

心智轉換對照表（ABAP ↔ SQLScript）：

ABAP	SQLScript	差異
DATA lv_x TYPE i.	DECLARE lv_x INTEGER ;	SQLScript 用資料庫原生型別（ INTEGER/DECIMAL/NVARCHAR 等），不是 ABAP DDIC 型別
lv_x = 1.	lv_x := 1;	純量變數賦值用 := （注意跟「查詢結果指派給表格變數」的 = 不同，見下方說明）
LOOP AT itab INTO wa.	FOR <記錄變數> AS <游標> DO ... END FOR ;	SQLScript 逐筆處理查詢結果要搭配游標（Cursor），不能像 ABAP 直接 LOOP AT 一個表格
IF ... ELSEIF ... ELSE ... ENDIF.	IF ... ELSEIF ... ELSE ... END IF ;	語意幾乎一樣，SQLScript 結尾是 END IF ；（有分號）
WHILE ... ENDWHILE.	WHILE ... DO ... END WHILE ;	這題沒用到，但語法結構類似 FOR

= 跟 **:=** 是兩個不同的賦值語意，這是最容易搞混的地方：**et_flights = SELECT ...**；這種寫法，右邊是一個查詢語句，**=** 代表「把這個查詢的結果集，指派給這個表格變數」；而 **lv_low_cnt := lv_low_cnt + 1**；這種寫法，右邊是一個純量運算式，**:=** 才是「一般程式語言那種賦值」。兩者不能混用（不能用 **:=** 接一個 **SELECT**，也不能用 **=** 接一個純量運算式）。

學習目標

- 會用 **DECLARE** 宣告 SQLScript 純量區域變數（資料庫原生型別，如 **INTEGER**），並用 **:=** 賦值
- 理解「逐筆處理查詢結果」在 SQLScript 裡要透過 **Cursor**（游標）：先 **DECLARE CURSOR** <游標名> **FOR** <**SELECT** 語句>; 再用 **FOR** <記錄變數> **AS** <游標名> **DO ... END FOR**；逐筆走訪，記錄變數的欄位用 **.** 存取（如 **cur_row.price**）
- 會寫 SQLScript 的 **IF/ELSEIF/ELSE/END IF**，語意跟 ABAP 的 **IF/ELSEIF/ELSE/ENDIF** 幾乎一樣，只是結尾語法不同
- 理解同一個分類需求（票價分成 **LOW/MEDIUM/HIGH**），宣告式（單一 **SELECT** 配 **CASE WHEN**）跟命令式（**DECLARE+FOR+IF**）兩種寫法並存，各自適合的場景不同

事前準備

ADT 端物件已由課程準備好 (`$TMP`) :

- `ZCL_AM03_PRICE_TIER`——AMDP 類別，`classify_flight_prices` 方法同時用兩種手法處理同一份資料：`et_flights` (明細+票價等級欄位) 用宣告式 `CASE WHEN` ;
`ev_low_cnt/ev_medium_cnt/ev_high_cnt` (三個等級各自的航班筆數) 用命令式的游標迴圈+`IF` 累加
- `ZR_AM03_DEMO`——demo 程式，帶 `p_carrid` (預設 `LH`)，呼叫後用 `WRITE` 印出明細與三個等級的統計

題目需求 (對照已建好的答案物件，含實測踩坑過程)

1. `ZCL_AM03_PRICE_TIER` 的簽章與 SQLScript 本體：

```
``abap CLASS zcl_am03_price_tier DEFINITION PUBLIC FINAL CREATE PUBLIC. PUBLIC SECTION. INTERFACES  
if_amdp_marker_hdb.
```

```
TYPES: BEGIN OF ty_flight_tier, carrid TYPE s_carr_id, connid TYPE s_conn_id, fldate TYPE s_date, price TYPE  
s_price, currency TYPE s_currcode, price_tier TYPE string, END OF ty_flight_tier, tt_flight_tier TYPE STANDARD  
TABLE OF ty_flight_tier WITH EMPTY KEY.
```

```
CLASS-METHODS classify_flight_prices IMPORTING VALUE(iv_mandt) TYPE mandt VALUE(iv_carrid) TYPE  
s_carr_id EXPORTING VALUE(et_flights) TYPE tt_flight_tier VALUE(ev_low_cnt) TYPE i VALUE(ev_medium_cnt)  
TYPE i VALUE(ev_high_cnt) TYPE i. ENDCLASS.
```

```
CLASS zcl_am03_price_tier IMPLEMENTATION.
```

```
METHOD classify_flight_prices BY DATABASE PROCEDURE FOR HDB LANGUAGE SQLSCRIPT OPTIONS READ-  
ONLY USING sflight.
```

```
DECLARE lv_low_cnt INTEGER := 0; DECLARE lv_medium_cnt INTEGER := 0; DECLARE lv_high_cnt INTEGER := 0;
```

```
DECLARE CURSOR cur_prices FOR SELECT price FROM sflight WHERE mandt = :iv_mandt AND carrid =  
:iv_carrid;
```

```
et_flights = SELECT carrid, connid, fldate, price, currency, CASE WHEN price < 300 THEN 'LOW' WHEN price <  
700 THEN 'MEDIUM' ELSE 'HIGH' END AS price_tier FROM sflight WHERE mandt = :iv_mandt AND carrid =  
:iv_carrid ORDER BY connid, fldate;
```

```
FOR cur_row AS cur_prices DO IF cur_row.price < 300 THEN lv_low_cnt := lv_low_cnt + 1; ELSEIF cur_row.price  
< 700 THEN lv_medium_cnt := lv_medium_cnt + 1; ELSE lv_high_cnt := lv_high_cnt + 1; END IF; END FOR;
```

```
ev_low_cnt := lv_low_cnt; ev_medium_cnt := lv_medium_cnt; ev_high_cnt := lv_high_cnt;
```

```
ENDMETHOD.
```

```
ENDCLASS. ``
```

1. 實測踩坑：`FOR <變數> AS <SELECT 語句>` 直接內嵌查詢不合法，第一版啟用失敗：

第一版嘗試把查詢直接寫在 `FOR ... AS` 後面 (省略獨立的 `DECLARE CURSOR`)：

```
``abap " 第一版・啟用失敗 FOR cur_row AS SELECT price FROM sflight WHERE mandt = :iv_mandt
AND carrid = :iv_carrid DO ... END FOR; ``
```

啟用時 HANA 編譯器直接回報：SQLSCRIPT: sql syntax error: incorrect syntax near "SELECT"，錯誤指向 `FOR cur_row AS SELECT` 這一行、剛好卡在 `SELECT` 這個字前面——**FOR <記錄變數> AS <名稱> DO 的 <名稱>** 必須是一個已經宣告好的 **Cursor** 名稱，不能直接塞一句 **SELECT**。修正方式是先用 `DECLARE CURSOR cur_prices FOR SELECT ...;` 把查詢包成一個具名游標，`FOR cur_row AS cur_prices DO` 才合法——這跟 ABAP 完全不一樣的地方在於：ABAP 的 `LOOP AT itab` 可以直接對著一個既有的 Internal Table 迴圈，不需要先幫它取名字；但 SQLScript 要逐筆走訪一個查詢結果，必須先把這個查詢「具名化」成一個 Cursor。

1. `ZR_AM03_DEMO` 呼叫端：

```
``abap REPORT zr_am03_demo.
```

```
PARAMETERS p_carrid TYPE s_carr_id DEFAULT 'LH'.
```

```
START-OF-SELECTION.
```

```
zcl_am03_price_tier=>classify_flight_prices( EXPORTING iv_mandt = sy-mandt iv_carrid = p_carrid IMPORTING
et_flights = DATA(lt_flights) ev_low_cnt = DATA(lv_low_cnt) ev_medium_cnt = DATA(lv_medium_cnt)
ev_high_cnt = DATA(lv_high_cnt) ).
```

```
WRITE: / 'FLIGHTS (with tier):'. LOOP AT lt_flights ASSIGNING FIELD-SYMBOL(<ls_flight>). WRITE: / <ls_flight>-
carrid, <ls_flight>-connid, <ls_flight>-fdate, <ls_flight>-price, <ls_flight>-currency, <ls_flight>-price_tier.
ENDLOOP.
```

```
WRITE: / 'TIER COUNTS:'. WRITE: / 'LOW =', lv_low_cnt. WRITE: / 'MEDIUM=', lv_medium_cnt. WRITE: / 'HIGH
=', lv_high_cnt. ``
```

預期輸出 (`p_carrid = 'LH'`，實測畫面節錄)

```
FLIGHTS (with tier):
LH  0400 2018/09/30      666.00  EUR   MEDIUM
...
LH  0400 2027/01/01      999.99  EUR   HIGH
LH  2402 2018/09/30      242.00  EUR   LOW
...
TIER COUNTS:
LOW   =           30
MEDIUM=          45
HIGH  =           1
```

`LOW + MEDIUM + HIGH = 30 + 45 + 1 = 76`，跟 `et_flights` 明細總筆數一致——兩種手法 (宣告式 **CASE WHEN** 產出的明細欄位、命令式游標迴圈累計的筆數) 算出來的結果互相對得上，這是驗證程式邏輯正確的好方法：同一份資料用兩種不同寫法各自算一次，結果應該一致。

團隊實務備註

- **這題刻意讓 `et_flights` (宣告式) 和 `ev*_cnt` (命令式) 在同一個方法裡處理同一份資料，是為了讓學員直接比較兩種寫法：如果只是要「給每筆資料加一個分類欄位」，宣告式的 `CASE WHEN` 一行搞定、效能通常也更好 (資料庫引擎最佳化過的集合運算)；但如果邏輯複雜到沒辦法用一句 `SQL` 表達 (例如要看前一筆的值才能決定這一筆怎麼處理、或是要呼叫其他程序做多步驟判斷)，才需要命令式的游標迴圈。不要看到這題就以為「以後迴圈都要這樣寫」——能一句 `SQL` 解決的，通常不需要迴圈**
- `FOR ... AS <cursor_name>` 一定要先 `DECLARE CURSOR`，這是這題唯一真正卡住啟用的地方：如果你以後看到其他 `SQLScript` 範例把 `SELECT` 直接寫在 `FOR ... AS` 後面沒有另外宣告 `Cursor`，很可能是不同資料庫版本/方言的差異寫法，這套系統 (HANA 2.0 SPS04) 實測是不允許的，一律先宣告 `Cursor` 再用名稱引用
- 游標裡的欄位用 `.` 存取 (`cur_row.price`)，不需要冒號：——冒號：只在「`SQLScript` 變數要放進一段內嵌 `SQL` 語句裡」時才需要 (例如 `WHERE mandt = :iv_mandt`)，`cur_row.price` 是在純程序邏輯 (`IF` 條件) 裡引用游標記錄的欄位，不是在寫一段 `SQL`，所以不加冒號——這個「什麼時候要加冒號、什麼時候不用」的判斷，是初學 `SQLScript` 最容易搞混的地方之一

思考題

1. 如果把 `DECLARE CURSOR cur_prices FOR SELECT price FROM sflight WHERE mandt = :iv_mandt AND carrid = :iv_carrid`; 改成直接 `SELECT price, connid FROM sflight ...` (多撈一個欄位)，`FOR cur_row AS cur_prices DO` 迴圈裡除了 `cur_row.price` 還可以用 `cur_row.connid` 嗎？
2. 這題的三個計數器 (`lv_low_cnt/lv_medium_cnt/lv_high_cnt`) 都是在 `SQLScript` 裡累加，如果改成單一個 `SELECT price_tier, COUNT(*) FROM (...) GROUP BY price_tier` 的宣告式寫法 (重用 `am02` 教過的聚合手法)，能不能得到一樣的三個數字？兩種寫法你會選哪一種，為什麼？
3. `WHILE` 迴圈這題沒有用到，只在 `Lecture` 對照表提過語法長相——如果要用 `WHILE` 改寫這題的游標迴圈 (`WHILE` 通常搭配一個判斷「還有沒有下一筆」的條件，而不是像 `FOR` 自動走訪整個游標)，你覺得程式碼會變得更簡單還是更複雜？這也是為什麼「逐筆走訪一個查詢結果」通常選 `FOR` 而不是 `WHILE` 的原因。

答案

見 `zcl_am03_price_tier.clas.abap`、`zr_am03_demo.prog.abap` (SAP 端物件 `ZCL_AM03_PRICE_TIER / ZR_AM03_DEMO`，套件 `$TMP`)。